You could implement this step as shown here:

```
Persona persona;                                          <-- Persona to be used for this scenario
@Given("name is $age years old")
public void givenAFrequentFlyerMemberOfAGivenAge(String name, int age) {
    persona = Persona.withName(name);                     <-- Look up persona by name
    persona.setAge(age);                                  <-- Override persona's age
}
```

Here you're simply starting off with the default field values for this persona and overriding the fields you want to change. This is very useful for scenarios where there are only slight variations in the data to be used.

A similar approach can be used with other types of test data. The team defines *known entities*—domain objects in a well-known state. For example, a banking application working on transferring bank statement files might define a few standard file types, and then only specify the fields that have different values for a given scenario.

7.3 Separating the what from the how

Automated acceptance tests need to be stable and reliable. When small changes happen in the application, you shouldn't need to update hundreds of acceptance tests just to keep the test suite running. Maintaining your automated acceptance tests should never be a hindrance to your ability to embrace change.

Yet when many teams start automating their acceptance criteria, this is exactly what happens. Often a small change in a single commonly used web page will break a large swath of tests, and developers will need to fix each test individually before the test suite can work again. This is thankless, unproductive work that does little to encourage the team to automate more acceptance tests.

It doesn't have to be this way. The trick is to apply a basic principle of software engineering, known as *layers of abstraction*. Layers of abstraction hide implementation details inside a function or object. This makes the high-level layers cleaner and easier to understand and also isolates them from changes in the implementation details hidden inside the lower layers.

When you write automated acceptance criteria, using layers can help you isolate the more volatile, low-level implementation details of your tests from the higher level, more stable business rules. High-level business rules tend to be relatively stable, and changes to them will be driven by the business rather than by technical constraints. Lower-level implementation details, such as screen layouts, field names, and how a low-level library is called, tend to change more frequently. When changes do happen at the lower implementation levels, the impact on the automated acceptance criteria should be minimal.

Experienced BDD practitioners[3] typically use at least three layers of abstraction in their tests, divided into three categories, like the ones illustrated in figure 7.8.

[3] See, for example, Gojko Adzik, "How to implement UI testing without shooting yourself in the foot," http://gojko.net/2010/04/13/how-to-implement-ui-testing-without-shooting-yourself-in-the-foot-2/.